

Wunomo AI Backend Product Documentation

Executive Summary

Wunomo AI is an AI-native platform designed to deliver specialized business personalities that function like digital operators for modern teams. Instead of being a single generic assistant, Wunomo AI is structured as a multi-personality product ecosystem where each personality is purpose-built for a specific operational domain such as data operations, analytics, governance, compliance, revenue operations, and executive decision support.

The first personality developed in Wunomo AI is the **DataOps Agent**. This personality has been built as a production-grade backend system that combines data source management, pipeline orchestration, quality control, incident handling, governance foundations, transformation execution, and conversational AI workflows into one unified operating layer.

This document provides a complete A-to-Z overview of the backend: what the product is, what has been built, what the DataOps Agent can do, how users flow through the system, what business value it creates, and why it has strong market relevance.

Product Description

Item	Description
Product Name	Wunomo AI
Product Type	AI-native multi-personality business operations platform
First Personality Built	DataOps Agent
Product Vision	Build a platform of domain-specific AI personalities that can operate business systems through natural language, governed execution, and modular backend services
Core Value Proposition	Replace fragmented operational tooling with intelligent, secure, explainable, and workflow-aware AI systems
Current Backend Status	First personality backend completed as a production-grade foundation

Platform Vision

Wunomo AI is designed as a long-term AI workforce platform. The idea is not to create only one assistant, but to create a system where multiple expert personalities can be launched under one product umbrella. Each personality can have its own tools, workflows, system permissions, operating style, and business logic while still sharing a common architecture pattern.

This makes Wunomo AI strategically powerful because one successful personality becomes the blueprint for building many more. The DataOps Agent is the first implementation of that

blueprint.

Personalities Wunomo AI Can Provide

The following personalities can be built and offered within the Wunomo AI product line.

No.	Personality	Primary Purpose
1	DataOps Agent	Manage data infrastructure, pipelines, quality, transformations, incidents, and operational trust
2	Analytics Agent	Explore metrics, explain trends, and support analytical workflows
3	BI Agent	Assist with dashboards, reports, semantic metrics, and executive reporting
4	Data Quality Agent	Focus on validation, quality rules, trust scoring, and data health monitoring
5	Governance Agent	Manage lineage, approvals, contracts, policy controls, and auditability
6	Incident Response Agent	Detect, triage, explain, and coordinate response to operational failures
7	ETL / Pipeline Engineer Agent	Build, adjust, optimize, and manage data pipeline operations
8	Knowledge Ops Agent	Organize internal knowledge, documentation, metadata, and process memory
9	RevOps Agent	Support CRM integrity, attribution quality, revenue workflows, and reporting reliability
10	Finance Ops Agent	Assist with reconciliation, controls, finance reporting, and operational close workflows
11	Support Ops Agent	Improve support SLAs, routing, service metrics, and support reporting quality
12	Compliance Agent	Enforce policy workflows, evidence logging, controls testing, and audit readiness
13	Growth Agent	Help monitor funnels, experiments, campaign quality, and growth KPI trust
14	Founder Copilot	Translate operational data into strategic summaries, risk signals, and executive decisions
15	Custom Enterprise Personalities	Industry-specific and organization-specific AI operators tailored to business needs

1. DataOps Agent

Overview

The DataOps Agent is the first personality built inside Wunomo AI. It is a production-grade backend platform designed to help teams operate data systems more intelligently. It gives users the ability to manage sources, inspect schemas, create pipelines, run validations, monitor reliability, handle incidents, govern risky actions, and generate transformations through a single unified backend.

Instead of forcing teams to jump between separate schedulers, notebooks, SQL tools, validation systems, incident trackers, and undocumented scripts, the DataOps Agent brings those workflows together into one AI-native operating layer.

What the DataOps Agent Is

The DataOps Agent acts as three systems at once:

Role	Meaning
System of Record	Stores operational data about sources, pipelines, runs, rules, incidents, approvals, contracts, lineage, KPIs, and chat actions
Execution Engine	Runs ingestion, orchestration, validations, transformations, and controlled operational actions
AI Reasoning Layer	Converts natural-language intent into tool-driven operational workflows with safety and governance controls

This combination is what makes the product more than a backend API. It becomes an intelligent operations platform.

Product Objective

The motive behind the DataOps Agent is to reduce the complexity, slowness, and risk that typically exist in data operations. In most organizations, data teams depend on too many disconnected systems: scripts, cron jobs, notebooks, monitoring dashboards, SQL editors, spreadsheets, alerts, and undocumented tribal knowledge.

That creates several real problems:

- Slow issue detection
- Slow root-cause analysis
- Repetitive manual work
- Weak governance
- Limited explainability for non-technical stakeholders
- High dependence on a few key engineers
- Reduced trust in downstream reporting and business decisions

The DataOps Agent is meant to solve this by making data operations more conversational, more observable, more governed, and more automatable.

Product Positioning

Dimension	Positioning
Category	AI-native DataOps platform
Primary Users	Data engineers, analytics engineers, platform teams, operations teams, founders, analysts, auditors

Dimension	Positioning
Interaction Style	API-first plus conversational AI workflows
Product Style	Multi-tenant, modular, governed, execution-capable backend
Core Differentiator	Combines orchestration, quality, governance, transformation, and AI interaction in one system
Long-Term Strategic Role	First personality and platform wedge for the full Wunomo AI ecosystem

High-Level Capabilities

The DataOps Agent backend is capable of the following :

Capability Area	What It Does
Authentication	Handles secure user registration, login, and user identity flows
Tenant Isolation	Ensures all tenant data remains logically separated
Source Management	Registers and manages databases and file-based sources
Schema Profiling	Captures schema snapshots and supports drift detection
Pipeline Orchestration	Creates, updates, schedules, triggers, pauses, and backfills pipelines
Run Tracking	Stores execution history, statuses, durations, outputs, and failures
Quality Control	Runs technical validation rules on data assets
Business Rules	Executes higher-level business integrity checks across sources and KPIs
Incident Management	Tracks operational failures, severity, root cause, and remediation
Governance Foundation	Supports lineage, approvals, contracts, and auditability
Transformation Engine	Generates and executes SQL and pandas transformations safely
Conversational Interface	Enables chat-driven operational workflows through AI orchestration

Backend Architecture Summary

Architecture Goals

The backend was designed with the following goals in mind:

- Multi-tenant SaaS readiness
- Strong operational safety
- AI-native user interaction
- Modular extensibility
- Production-oriented execution controls
- Clear separation of responsibilities between modules

- API-first product architecture

Core Stack

Layer	Technology / Design Choice
API Framework	FastAPI
Agent Orchestration	LangGraph and LangChain
Primary Database	PostgreSQL
Database Access	SQLAlchemy async + asyncpg
Queue / Async Support	Redis
Background Jobs	Celery
LLM Layer	Gemini primary path with fallback LLM support described in the earlier build context
Logging	structlog
Test Framework	pytest + pytest-asyncio
Deployment Pattern	Docker Compose multi-service backend

Service Layout

Service	Responsibility
Backend API	Main application layer and REST endpoints
PostgreSQL	Persistent system state
Redis	Background task and queue infrastructure
Celery Worker	Async execution of background jobs
Celery Beat	Scheduled task execution

Core Backend Modules

The backend is organized as modular capabilities rather than one large service block.

Module Area	Purpose
Authentication	User access, JWT flows, current-user context
Ingestion	Source registration, connection handling, previews
Schema Profiling	Source profiling and drift awareness
Orchestration	Pipelines, scheduling, triggering, backfill, run lifecycle
Quality	Technical validation rules and business rules
Observability	Reliability visibility and operational health functions
Governance	Lineage, auditability, contracts, approvals

Module Area	Purpose
Reporting / Analytics	Operational summaries, KPIs, metrics, reporting views
Transformation	SQL generation, pandas generation, safe execution
Agent Layer	Natural language orchestration across tools and workflows

Data Model Coverage

The backend includes a rich operational model set so the platform can remember and govern system activity over time.

Model	Business Purpose
Tenant	Organization boundary
User	Identity and permissions context
DataSource	Registered source metadata and connection settings
Pipeline	Operational definition of data workflows
PipelineRun	Execution history and result tracking
QualityRule	Technical validation logic
Incident	Operational issue tracking and response metadata
LineageNode	Assets in the lineage graph
LineageEdge	Relationships between lineage assets
AuditLog	Historical record of operational actions
ApprovalRequest	Governance control for risky actions
ChatMessage	Conversational memory for agent interactions
DataContract	Expected schema and quality agreements
UsageMetric	Product and system usage tracking
KpiValue	KPI storage for trend and monitoring use cases

User Interaction Model

The backend supports a flexible interaction model through **personality modes** and **operation modes**.

Personality Modes

Personality Mode	Purpose
Engineer	Technical diagnostics, root cause analysis, schema-aware execution
Founder	Executive summaries, risk framing, business impact communication
Analyst	KPI trust, freshness insight, dataset readiness, reporting context

Personality Mode	Purpose
Auditor	Lineage, traceability, approvals, and compliance visibility

Operation Modes

Operation Mode	Behavior
Advisory	Guidance only, no execution
Assisted	Low-risk execution with approvals for riskier actions
Autonomous	Expanded execution subject to governance safeguards
Audit	Read-only investigative and review mode

This interaction design is important because it lets the same backend behave differently depending on who is using it and how much execution authority should be allowed.

Core Product Features

Authentication and Tenant Control

The platform includes JWT-based authentication flows for registration, login, and user retrieval. Tenant isolation is treated as a foundational system rule, meaning the backend is architected to keep one tenant’s operational data separate from another’s.

Why it matters

- Supports SaaS readiness
- Enables secure multi-organization usage
- Creates a clean foundation for enterprise deployment

Source Management

The backend supports source registration and lifecycle management for both database and file-based systems.

Source support defined in the product context

Source Type Category	Examples
Relational Databases	PostgreSQL, MySQL, SQLite
Cloud Warehouses	BigQuery, Snowflake
File Sources	CSV, Excel, JSON
External / Connected Systems	REST API, Google Sheets, S3
Document Inputs	PDF, DOCX

Implemented and planned orientation

Status Type	Sources
Implemented in earlier delivered backend context	PostgreSQL, MySQL, CSV, Excel, JSON
Mentioned in broader source model / roadmap context	SQLite, BigQuery, Snowflake, REST API, Google Sheets, S3, PDF, DOCX

Source operations

- Register source
- Update source
- Deactivate source
- Preview source content
- Sync metadata
- Profile source schema
- Detect schema drift

Schema Profiling and Drift Detection

Schema profiling captures actual source structure and stores schema snapshots that can later be used by the AI system. This is especially important for transformation generation because SQL and pandas prompts become much more accurate when the model is shown real columns and types.

Benefits

- Better transformation accuracy
- Safer AI-generated logic
- Early detection of structural changes
- Better downstream trust in automation

Pipeline Lifecycle Management

The backend includes a complete orchestration layer for pipelines.

Pipeline operations

Action	Description
Create	Define a new pipeline
List / Get	Retrieve pipeline state and metadata
Update	Modify settings and configuration
Trigger	Start a pipeline run manually

Action	Description
Pause	Disable active execution
Activate	Re-enable pipeline activity
Schedule	Assign or update cron-based timing
Backfill	Re-run historical execution windows
History	Inspect prior run records

Pipeline design elements

- Source linkage
- Status management
- Retry policy
- SLA tracking
- Run metadata capture
- Historical state visibility

Run Tracking

Pipeline runs are modeled as first-class operational objects.

Run Attribute	Why It Matters
Status	Shows current execution state
Triggered By	Identifies user or system origin
Started / Completed At	Allows duration and timing analysis
Duration	Helps measure pipeline efficiency
Rows Processed / Failed	Quantifies execution output quality
Quality Score	Connects data trust to pipeline runs
Output Summary	Summarizes what happened
Error Message	Supports diagnosis and debugging
Retry Count	Tracks resilience behavior

This is valuable because the backend does not just execute pipelines; it creates historical operational memory.

Quality Layer

The DataOps Agent includes both **technical quality rules** and **business rules**.

Technical Quality Rules

Rule Type	Purpose
Not Null	Ensures required values exist
Unique	Prevents duplicate values where uniqueness is expected
Accepted Values	Restricts values to an allowed set
Range	Ensures numeric or logical bounds
Regex	Checks string pattern validity
Row Count	Validates expected row behavior
Custom SQL	Allows custom data checks

Business Rules Added in Phase 4

Business Rule	What It Checks
Reconciliation	Source A and Source B match within tolerance
KPI Sanity	KPI remains inside allowed thresholds
Cross-Source Count	Record count consistency across sources
Freshness SLA	Source is updated within expected time window
Column Sum Match	Aggregated values align across systems
KPI Trend	KPI does not degrade unexpectedly over time
Pipeline Success Rate	Operational success stays above threshold
Data Volume Growth	Volume growth remains within expected range

Why this matters

Technical rules protect data structure and validity. Business rules protect business meaning. Together, they create a stronger trust system than either layer could provide alone.

Incident Management

Incidents are represented as dedicated backend entities so failures can be treated operationally rather than informally.

Incident Field	Purpose
Severity	Indicates urgency
Status	Tracks open, investigating, resolved, or suppressed state

Incident Field	Purpose
Root Cause	Captures underlying issue narrative
Remediation Actions	Documents how resolution should occur
Affected Assets	Shows business and system blast radius
Detected At / Resolved At	Measures response speed
Resolution Notes	Preserves resolution knowledge

Value

- Moves error handling into a formal response system
- Improves institutional memory
- Helps connect technical failure to business impact

Governance Foundation

The backend includes structures and workflows that support governed AI operations.

Governance capabilities

Capability	Role in the System
Lineage	Represents upstream and downstream asset relationships
Audit Logs	Preserves action history for accountability
Approval Requests	Adds human review for risky operations
Data Contracts	Captures expected schema and quality commitments
Risk Modes	Shapes how much the agent is allowed to do

Why it matters

This is a key step toward enterprise readiness. AI systems become much more acceptable inside organizations when their actions are visible, reviewable, and governable.

Transformation Engine

The transformation engine is one of the most important additions because it allows the DataOps Agent to move beyond monitoring and into active data shaping.

Main components

Component	Purpose
Transform Generator	Converts user requests into SQL or pandas logic
SQL Runner	Executes safe SQL with multiple protective layers

Component	Purpose
Python Runner	Executes restricted pandas code inside a sandbox

Transform Generator

The transform generator acts as a unified natural-language transformation interface.

Included capabilities

- Unified `generate()` entry point
- SQL generation for database-backed sources
- Pandas generation for file-backed sources
- Explicit SQL and pandas generation methods
- Schema snapshot injection into prompts
- Code explanation in plain English
- Post-generation validation checks

Validation and safety checks

Area	Protection
SQL	Blocks destructive patterns like DROP, TRUNCATE, and DELETE
SQL	Warns when LIMIT is missing
Pandas	Blocks risky functions and unsafe imports
Pandas	Ensures <code>result_df</code> assignment exists

SQL Runner

The SQL runner is designed for practical execution with strong safety protections.

SQL safety layers

Safety Layer	Description
Keyword Blocklist	Blocks dangerous SQL commands
Select-Only Enforcement	Requires safe query entry patterns
Automatic LIMIT Injection	Prevents unbounded queries by default
Hard Row Cap	Ensures outputs remain controlled
Audit Logging	Records execution events

Execution methods

Method	Purpose
<code>run_on_source()</code>	Executes SQL against a registered data source
<code>run_internal()</code>	Executes internal system SQL use cases
<code>dry_run()</code>	Returns EXPLAIN-only plan data

Python Runner

The Python runner gives the system a controlled dataframe transformation path.

Isolation design

Protection Type	What It Does
AST Static Analysis	Examines code before execution
Import Restrictions	Limits imports to approved libraries
Name Blocking	Prevents dangerous names and functions
Attribute Blocking	Prevents risky object access
Restricted Builtins	Removes unsafe execution capabilities
Execution Timeout	Stops long-running tasks
Output Controls	Caps rows and serializes results safely

Why it matters

This allows the product to provide notebook-like transformation value without exposing the backend to unrestricted Python execution risk.

API Surface

The backend is API-first, which is important for frontend development, product integration, and enterprise extensibility.

Major API areas

API Area	Purpose
Auth	Register, login, current user
Sources	CRUD, profiling, sync, preview, drift
Pipelines	CRUD, trigger, pause, activate, schedule, history, backfill
Quality	Create, list, run, and delete validation rules
Incidents	Create, inspect, update, and resolve issues

API Area	Purpose
Governance	Lineage, contracts, audit functions
Approvals	Review and act on pending approval requests
Analytics	Operational metrics and KPI access
Chat	Conversational agent interaction
Transformations	Generate, preview, explain, dry-run, and execute SQL / pandas

Transformation endpoints introduced in Phase 4

Endpoint Type	Purpose
Generate	Auto-select SQL or pandas
Generate SQL	Explicit SQL generation
Generate Pandas	Explicit pandas generation
Run SQL	Execute safe SQL
Dry-Run SQL	Return EXPLAIN plan only
Run Pandas	Execute sandboxed pandas transformation
Preview Pandas	Run preview on limited rows
Explain	Convert code to plain-language explanation

Conversational AI Workflow

The DataOps Agent is designed to operate through chat-driven workflows in addition to direct API interaction. A user can state an operational goal in natural language, and the backend can interpret that request, select the right tools, apply the user's operating mode, and determine whether execution, explanation, preview, or approval is appropriate.

This is a major product advantage because it lowers the barrier between human intent and operational execution.

End-to-End User Flow

The following table shows how a user moves through the product from start to finish.

Step	User Journey	System Outcome
1	User registers and logs in	Secure tenant-scoped access is created
2	User adds a data source	Source metadata and connection details are stored
3	User profiles the source	Schema snapshot is generated and saved
4	User creates a pipeline	Pipeline object and operational settings are defined
5	User adds quality rules	Technical and business trust checks are configured

Step	User Journey	System Outcome
6	User triggers or schedules runs	Execution history begins to accumulate
7	System records run outcomes	Operational visibility improves over time
8	User asks for transformation help	SQL or pandas logic is generated from natural language
9	User previews or executes safely	Controlled transformation output is produced
10	Incidents or anomalies are investigated	Root-cause workflows become structured
11	Approvals and audit controls apply when needed	Governance is preserved
12	Product reports trust, health, and impact	Teams get both technical and business-level visibility

Example User Personas

User Type	How They Benefit
Data Engineer	Faster diagnostics, safer transformations, centralized operations
Analytics Engineer	Better schema visibility, quality checks, and trusted datasets
Founder / Executive	High-level summaries, risk awareness, and business impact framing
Auditor / Governance Lead	Better traceability, approvals, and system accountability
Operations Team	Faster issue coordination and operational visibility

Business Value

Operational Value

Value Area	Impact
Speed	Reduces time to inspect sources, pipelines, and issues
Trust	Improves confidence in data assets and outputs
Safety	Adds controls around transformations and risky actions
Efficiency	Reduces repetitive operational work
Explainability	Converts technical state into understandable summaries
Scalability	Creates reusable system patterns for growing organizations

Strategic Value

Strategic Benefit	Why It Matters
Platform Wedge	Data operations is a strong first entry point into organizations
Reusable Pattern	Same architecture can power future personalities

Strategic Benefit	Why It Matters
Expansion Potential	Can later extend into analytics, governance, finance ops, rev ops, and founder intelligence
Enterprise Readiness	Governance and control features improve credibility in serious environments

Market Need

There is strong market demand for systems that reduce operational complexity around data infrastructure. Modern teams rely heavily on data for reporting, automation, internal tools, AI systems, and executive decision-making, but the operational tooling around that data is often fragmented and reactive.

Common market pain points

- Too many disconnected tools
- Limited reliability visibility
- Slow issue response cycles
- Poor governance of operational actions
- Low explainability for business teams
- Heavy dependence on internal specialists

The DataOps Agent addresses these issues by acting as an AI-native control layer on top of the data stack.

Market Positioning Potential

Positioning Angle	Commercial Relevance
AI DataOps Platform	Appeals to modern data teams seeking automation and visibility
Data Reliability Layer	Useful for trust, quality, and reporting confidence
Governed AI Operator	Valuable for organizations that need reviewable AI actions
Multi-Personality Platform	Expands revenue potential beyond one use case
Enterprise Operational Copilot	Strong positioning for internal platform and ops teams

Product Impact

If deployed successfully, the DataOps Agent can create meaningful impact for organizations.

Impact Area	Expected Result
Reliability	Faster identification of broken or degraded data workflows
Productivity	Less manual work for engineering and operations teams
Decision Quality	Better trust in downstream reporting and KPIs
Governance	More visibility into who did what and why

Impact Area	Expected Result
Accessibility	More non-specialist users can interact with data operations through natural language
Institutional Knowledge	Operational context becomes captured in the platform instead of living only in people's heads

What Has Been Built

The backend foundation that has been built represents a full product core rather than a disconnected codebase.

Built product areas

Area	Status in Product Story
Product identity and first personality	Built
Authentication and tenant model	Built
Source registration and profiling	Built
Pipeline orchestration system	Built
Run tracking	Built
Quality rule engine	Built
Business rules engine	Built
Incident management model	Built
Governance foundation	Built
AI interaction layer	Built
Transformation generation and execution	Built
Multi-service deployment structure	Built

Why This Backend Matters

The importance of this backend is larger than the feature list itself. It is the first working proof that Wunomo AI can turn a business function into a personality-driven, tool-using, safety-aware, production-ready AI system.

That makes the DataOps Agent important in two ways:

1. As a standalone product for data operations.
2. As the architecture blueprint for every future Wunomo AI personality.

Next Product Layer

With the backend foundation in place for the first personality, the next major product layer is the frontend experience. That frontend can expose dashboards, chat interfaces, pipeline timelines, governance views, lineage graphs, incident views, and approval workflows on top of the backend that has already been established.

Final Perspective

Wunomo AI is not simply building another generic assistant. It is building a platform of intelligent operational personalities. The DataOps Agent is the first and most important proof of that model. It already demonstrates how AI can be combined with APIs, governance, transformations, operational memory, and modular backend systems to create a practical enterprise-ready product foundation.

For that reason, this backend is both the first product and the first repeatable platform template for the broader Wunomo AI vision.